

Software Design Document

Tài liệu Thiết kế

Khởi tổng hợp dữ liệu

(STM32 + FreeRTOS)

Mã tài liệu:

Tổng số trang: 17 trang

MỤC LỤC

BẢNG GHI NHẬN THAY ĐỔI	4
DANH MỤC HÌNH ẢNH	5
1. Giới thiệu.....	6
1.1. Mục đích tài liệu	6
1.2. Phạm vi tài liệu	6
1.3. Khái niệm, thuật ngữ, viết tắt	6
2. Mô tả tổng quan.....	8
2.1. Sơ đồ tổng quan hệ thống	9
2.2. Mô hình người dùng và ngữ cảnh hoạt động.....	9
3. Yêu cầu về chức năng.....	11
4. Kiến trúc phần mềm và các thành phần chính.....	12
4.1. ISR UART RX	13
4.2. Buffer UART RX.....	13
4.3. Task UART Rx Parser.....	13
4.4. Queue UART3 Tx	14
4.5. Task UART3 Tx	14
4.6. Txcppltcallback.....	15
5. Chi tiết xử lý MAVLink V1	15
6. Kết quả thử nghiệm hoạt động.....	15
7. Một số lưu ý khác	17
8. Kết luận	17

BẢNG GHI NHẬN THAY ĐỔI

[illegible]

DANH MỤC HÌNH ẢNH

Hình 1 – Sơ đồ tổng quan hệ thống “Khởi tổng hợp dữ liệu”

Hình 2 – Demo dự án trên thực tế

Hình 3 – Sơ đồ luồng dữ liệu của thiết bị

Hình 4 – Sơ đồ thuật toán hoạt động của “Khởi tổng hợp dữ liệu”

Hình 5 – Kết quả thử nghiệm khi bật thiết bị

Hình 6 – Kết quả thử nghiệm khi truyền tin MAVLink vào cổng UART2

Hình 7 – Kết quả thử nghiệm khi truyền tin MAVLink vào 2 cổng UART1 và UART2

1. Giới thiệu

Tài liệu thiết kế “Khởi tổng hợp dữ liệu” là cơ sở triển khai dự án lập trình nhúng cho thiết bị có chức năng tổng hợp dữ liệu từ nhiều luồng khác nhau cùng một lúc để xử lý rồi xuất dữ liệu ra màn hình điều khiển. Tài liệu cung cấp mô tả chi tiết các thành phần, chức năng và luồng dữ liệu trong thiết bị được triển khai.

Trong dự án sử dụng vi điều khiển STM32 có ít nhất 3 cổng UART vật lý và sử dụng hệ điều hành thời gian thực FreeRTOS. Mục tiêu của dự án là thu thập dữ liệu từ 2 nguồn UART, kiểm tra xem dữ liệu có phải bản tin MAVLink V1 không, và nếu đúng thì chuyển bản tin đó qua cổng UART đầu ra.

1.1. Mục đích tài liệu

Tài liệu này nhằm:

- Cung cấp mô tả liên kết chức năng giữa các thành phần chính của thiết bị: STM32 với USB-UART.
- Cung cấp những lưu ý trong thiết kế và lắp đặt các thành phần của thiết bị.
- Cung cấp các thông số kỹ thuật để hỗ trợ việc bảo trì, nâng cấp và phát triển.

Đối tượng sử dụng tài liệu cụ thể được nêu ở trong bảng 1.

Bảng 1 – Đối tượng sử dụng tài liệu

Người sử dụng	Mục đích
Người dùng	Hiểu rõ nguyên lý hoạt động và cấu trúc hệ thống
Nhóm phát triển	Hiểu rõ các thông số kỹ thuật, chức năng và đặc tính của hệ thống để phục vụ cho việc bảo trì, khắc phục lỗi và phát triển hệ thống theo yêu cầu của người dùng
Nhóm kiểm thử	Hỗ trợ xây dựng tài liệu kiểm tra hoạt động của thiết bị
SQA	Sử dụng để tham chiếu kiểm tra hoạt động của thiết bị, kiểm tra các tài liệu của nhóm phát triển và kiểm thử

1.2. Phạm vi tài liệu

Tài liệu này được sử dụng trong phạm vi dự án thiết kế “Khởi tổng hợp dữ liệu”.

1.3. Khái niệm, thuật ngữ, viết tắt

Bảng 2 – Khái niệm, thuật ngữ và viết tắt

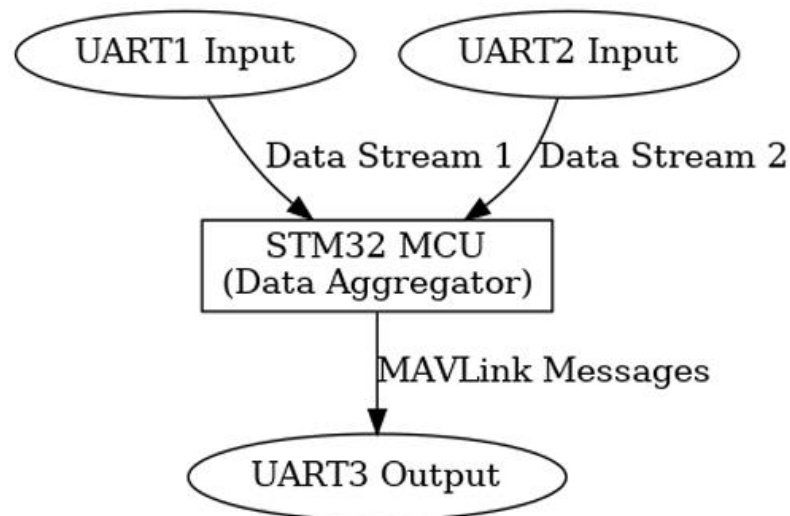
ID	Acronyms	English Term	Definition
1	Buffer	Buffer	Bộ đệm dùng lưu tạm thời dữ liệu trong quá trình xử lý
2	CMSIS-RTOS	Cortex Microcontroller Software Interface Standard – RTOS	Chuẩn giao diện RTOS do ARM phát triển, hỗ trợ FreeRTOS trên STM32.
3	COM	Communication Port	Cổng nối tiếp (Serial Port) dùng để giao tiếp giữa máy tính và thiết bị.

4	CPU	Central Processing Unit	Bộ xử lý trung tâm, thực thi lệnh và điều khiển toàn bộ hoạt động của hệ thống.
5	DMA	Direct Memory Access	Truy cập bộ nhớ trực tiếp mà không cần CPU can thiệp.
6	FIFO	First In First Out	Bộ đệm kiểu hàng đợi vào trước ra trước, giúp xử lý luồng dữ liệu nối tiếp.
7	FreeRTOS	Free Real-Time Operating System	Hệ điều hành thời gian thực mã nguồn mở dành cho vi điều khiển.
8	IDLE	Idle Line Detection	Trạng thái rảnh của UART, báo hiệu kết thúc dòng truyền.
9	ISR	Interrupt Service Routine	Hàm phục vụ ngắt, xử lý sự kiện xảy ra tức thời.
10	MAVLink	Micro Air Vehicle Link	Giao thức truyền thông nhẹ dùng cho drone và robot.
11	MCU	Microcontroller Unit	Vi điều khiển – chip tích hợp CPU, RAM, ROM và ngoại vi.
12	PC	Personal Computer	Máy tính cá nhân – thiết bị trung tâm giao tiếp và lập trình.
13	Parse	Parse	Phân tích cú pháp – trích xuất và hiểu cấu trúc dữ liệu.
14	Polling	Polling	Cách kiểm tra trạng thái ngoại vi liên tục bằng phần mềm.
15	Queue	Queue	Hàng đợi dùng để truyền dữ liệu giữa các task trong FreeRTOS.
16	Rx	Receive	Nhận dữ liệu (Receive) qua UART hoặc cổng truyền.
17	STM32	STMicroelectronics 32-bit Microcontroller	Vi điều khiển 32-bit của hãng STMicroelectronics.
18	STM32CubeIDE	STM32 Cube Integrated Development Environment	Môi trường lập trình tích hợp dành cho STM32.
19	STM32CubeMX	STM32 Cube Configuration and Initialization Tool	Công cụ cấu hình chân, ngoại vi và sinh mã khởi tạo cho STM32.
20	Semaphore	Semaphore	Cơ chế đồng bộ giữa các task hoặc giữa ISR và task.
21	Task	Task	Luồng thực thi độc lập trong FreeRTOS.
22	Tx	Transmit	Gửi dữ liệu (Transmit) qua UART hoặc giao tiếp nối tiếp.
23	Txcpltcallback	Transmit Complete Callback	Hàm được gọi khi DMA truyền UART hoàn tất.
24	UART	Universal Asynchronous Receiver Transmitter	Chuẩn truyền thông nối tiếp không đồng bộ.
25	USB TTL	USB to TTL (Transistor-Transistor Logic) Converter	Bộ chuyển đổi tín hiệu USB sang UART mức TTL (3.3V/5V).

Tài liệu tham khảo

ID	Reference	Version Number	Document Name/Source & Location reference	Describe	Remarks
1	KienNguyen		Lorenz Meier et al. MAVLink Protocol Documentation.		
2	KienNguyen		FreeRTOS.org. FreeRTOS Queue API.		
3	KienNguyen		STMicroelectronics. STM32 HAL Library– UART DMA Reception.		
4	KienNguyen		FreeRTOS.org. FreeRTOS Stream Buffer API.		
5			STM32 RTOS – GPIO Tutorial (CMSIS V2)		
6			Receive Data using IDLE LINE		

2. Mô tả tổng quan



Hình 1 – Sơ đồ tổng quan hệ thống “Khởi tổng hợp dữ liệu”

Sơ đồ tổng quan hệ thống “Khởi tổng hợp dữ liệu” thể hiện trực quan mối liên kết giữa các thành phần trong thiết bị và luồng dữ liệu được xử lý. Như đã thấy, thiết bị bao gồm 4 thành phần chính: Đầu vào được lấy từ 2 cổng UART - UART1 và UART2; dữ liệu sau đó được xử lý tại MCU và đưa ra ngoài tại cổng UART3.

2.1. Sơ đồ tổng quan hệ thống

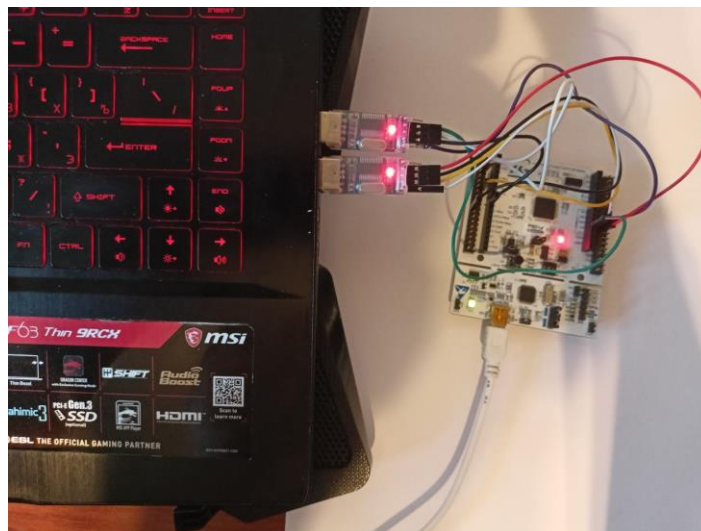
“Khối tổng hợp dữ liệu” được xây dựng và kiểm thử trên các module bán sẵn trên thị trường như sau:

a, Phần cứng:

- 1 Bo mạch gỡ lỗi Nucleo-F103RBT6: Có 3 cổng UART, hỗ trợ sử dụng CMSIS-RTOS V1/V2 (FreeRTOS của hãng STM32), DMA, ISR và tích hợp bộ lập trình ST-LINK/V2.
- 2 Bộ chuyển đổi USB sang UART (USB TTL) PL2303HX/PL2303.

b, Phần mềm

- STM32CubeMX: sử dụng để tạo các code cấu hình cho phần cứng của hãng STM32.
- STM32CubeIDE: sử dụng để xây dựng code xử lý các yêu cầu về chức năng của thiết bị.
- Visual Studio Code: sử dụng để tạo ra các gói tin MAVLink V1 gửi tới các cổng UART1/UART2 của thiết bị.
- terminal 1.9b: sử dụng để hiển thị các nội dung bản tin sau khi xử lý và đưa ra cổng UART3.



Hình 2 – Demo dự án trên thực tế

2.2. Mô hình người dùng và ngữ cảnh hoạt động

Trước khi tiến hành kiểm thử hoạt động của thiết bị “Khối tổng hợp dữ liệu”, người dùng cần thực hiện các thao tác lắp đặt sau:

- Kết nối các cổng UART trên bo mạch Nucleo-F103RBT6 với các bộ chuyển đổi USB TTL: UART1 với USB TTL №1, UART2 sử dụng trực tiếp bộ chuyển đổi trên bo mạch Nucleo-F103RBT6, UART3 với USB TTL №2.
- Kết nối các bộ chuyển đổi với PC, sau đó kiểm tra địa chỉ các cổng COM đã được kết nối trong “Device manager”.

- Thiết lập chính xác các cổng COM tương ứng với từng UART cho các mã nguồn sử dụng để gửi gói tin MAVLink V1 tới thiết bị và đưa ra trên terminal 1.9b.

- Cấu hình cho cả 3 cổng COM (UART):

- Baudrate: 115200;
- Data size: 8;
- Parity: None;
- Stopbit: 1;
- Handshake: None.

Sau khi hoàn thiện các tác vụ trên, người dùng có thể tiến hành kiểm thử hoạt động của thiết bị:

Bước 1: Mở terminal 1.9b và kết nối với cổng COM (UART3) để hiển thị các nội dung bản tin.

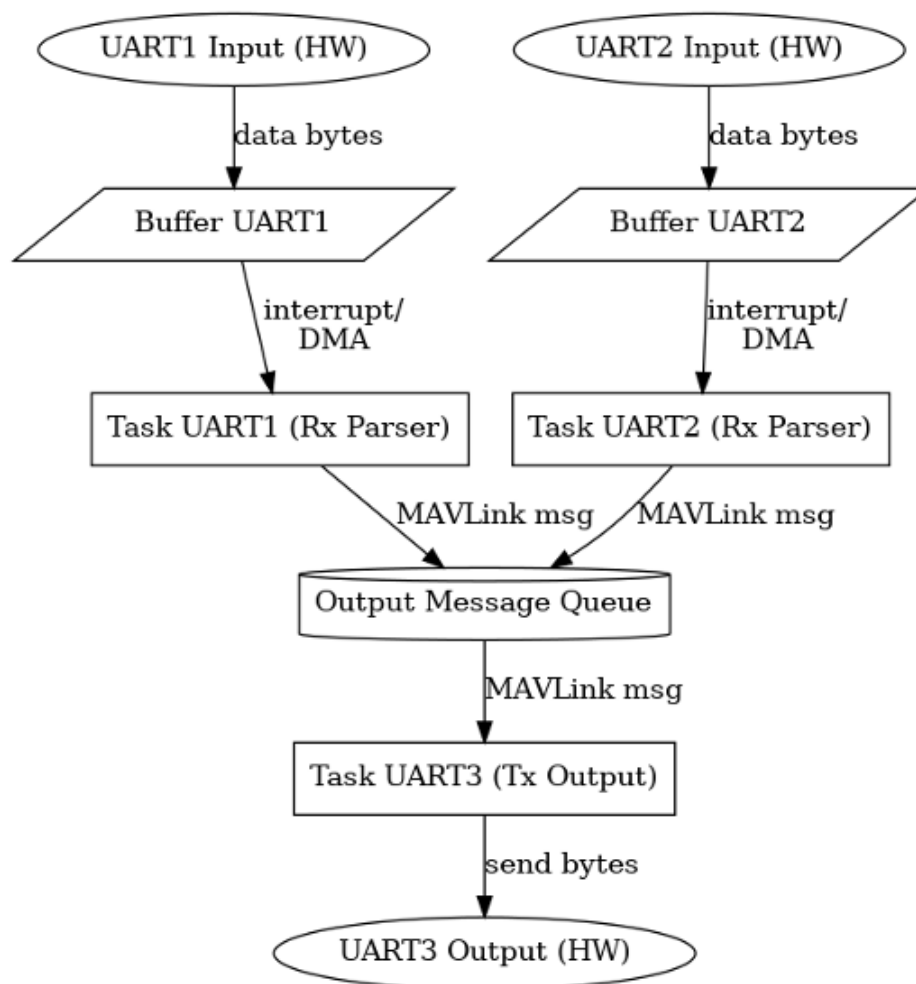
Bước 2: Nạp mã nguồn cho thiết bị trong môi trường STM32CubeIDE

Bước 3: Mở Visual Studio Code để chạy các chương trình tạo gói tin MAVLink V1 và gửi tới 2 cổng COM (của UART1 và UART2).

Bước 4: Theo dõi các nội dung được hiển thị trên terminal 1.9b

Người dùng có thể thay đổi nội dung bản tin, tốc độ gửi tin qua các cổng COM (UART1/UART2) trong đoạn mã nguồn của các chương trình tạo gói tin MAVLink V1.

3. Yêu cầu về chức năng



Hình 3 – Sơ đồ luồng dữ liệu của thiết bị

Dựa vào sơ đồ luồng dữ liệu ở hình 3, chúng ta có thể xác định được những chức năng cần có của thiết bị như sau:

- Thiết bị nhận gói tin tại 2 cổng UART1 và UART2 rồi lưu vào “Buffer UART1” và “Buffer UART2” tương ứng;
- Sau đó, dữ liệu là các gói tin MAVLink V1 được phân tích (parse) tại các Task UART1/UART2 (Rx Parser);
- Dữ liệu sau được parse sẽ được đưa vào Queue (Output Message Queue) để lần lượt gửi lên terminal 1.9b bằng Task UART3 (Tx Output) qua COM (UART3).

Toàn bộ việc gửi và nhận qua các UART được sử dụng với DMA và ISR để không làm tốn tài nguyên của CPU, trong thời gian đấy CPU có thể thực hiện các tác vụ khác.

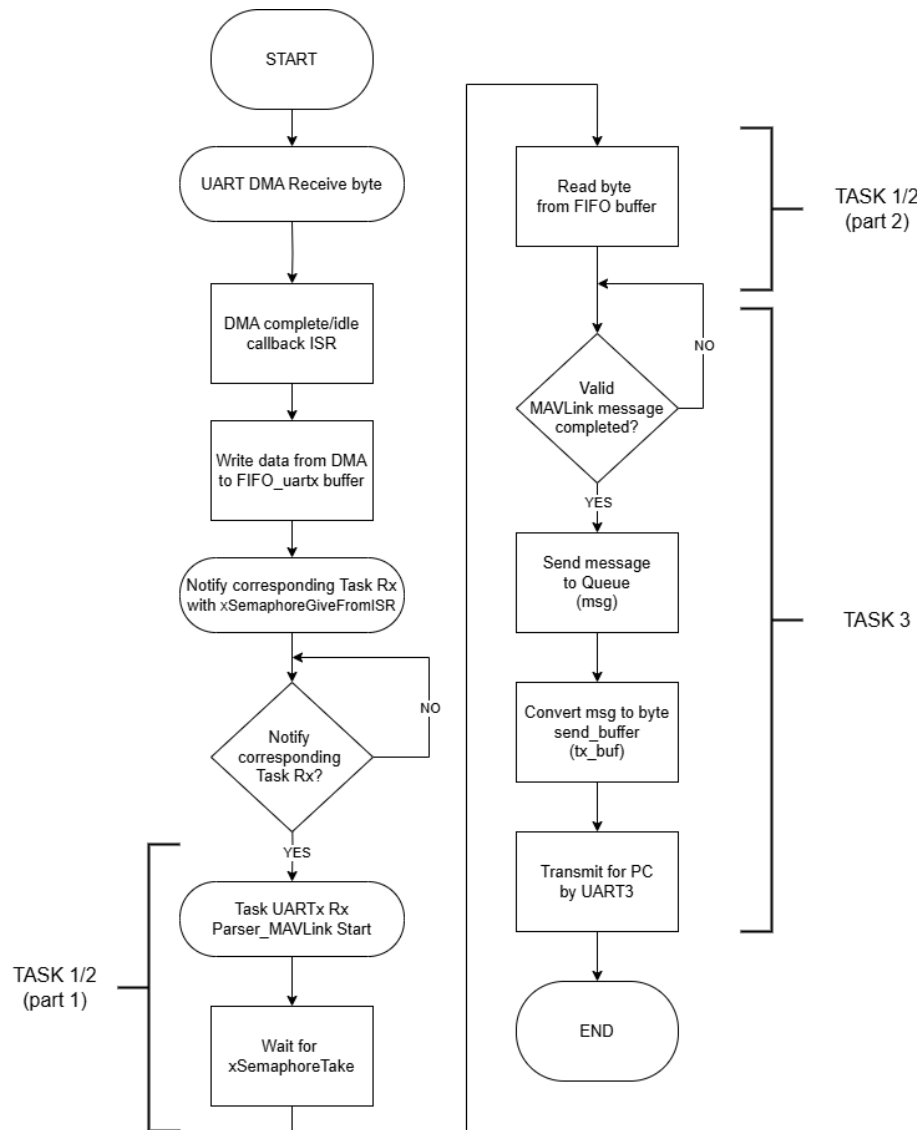
4. Kiến trúc phần mềm và các thành phần chính

Hệ thống sử dụng kiến trúc đa tác vụ trên CMSIS-RTOS V2 (FreeRTOS của hãng STM32), tận dụng các cơ chế hàng đợi (queue), bộ đệm (buffer) và ngắt (ISR) để xử lý song song dữ liệu từ 2 cổng UART.

Đồng thời sử dụng DMA cho phép các ngoại vi trực tiếp truy cập vào bộ nhớ mà không cần sự can thiệp của CPU, giúp việc truyền dữ liệu giữa thiết bị ngoại vi và bộ nhớ một cách độc lập với CPU. Từ đó tăng tốc độ truyền dữ liệu và giảm tải cho CPU.

Ngoài ra, để tối ưu việc thực hiện các tác vụ song song trên CMSIS-RTOS V2, sử dụng công cụ đồng bộ Semaphore để kiểm soát quyền truy cập tài nguyên chung giữa nhiều task khác nhau.

Dưới đây là sơ đồ thuật toán hoạt động của thiết bị “Khối tổng hợp dữ liệu”.



Hình 4 – Sơ đồ thuật toán hoạt động của “Khối tổng hợp dữ liệu”

Sau đây chúng ta sẽ phân tích chi tiết từng thành phần chính của thiết bị:

4.1. ISR UART RX

Khi có byte dữ liệu được gửi đến qua UART1/UART2, DMA tự động ghi vào vùng đệm vòng `uartx_dma_rx_buf` ($x = 1, 2$). Khi phát hiện ngắt IDLE (khoảng dừng giữa các byte), ISR `UARTx_IRQHandler()` sẽ được gọi.

Trong ISR tính toán số byte mới nhận, sau đó chép dữ liệu từ DMA vào FIFO (`uart1_fifo`), rồi đánh thức task xử lý (bằng `xSemaphoreGiveFromISR`) để tiếp tục xử lý và parse bản tin MAVLink V1 trong ngữ cảnh Task của CMSIS-RTOS.

Pseudocode minh họa 1 – ISR UART Rx

```
void USART1_IRQHandler(void)
{
    if (__HAL_UART_GET_FLAG(&huart1, UART_FLAG_IDLE))
    {
        __HAL_UART_CLEAR_IDLEFLAG(&huart1);

        static uint16_t last_dma_pos_uart1 = 0;

        uint16_t dma_pos = UART_RX_BUFFER_SIZE - __HAL_DMA_GET_COUNTER(&hdma_usart1_rx);
        uint16_t len = (dma_pos >= last_dma_pos_uart1) ? (dma_pos - last_dma_pos_uart1) :
        (UART_RX_BUFFER_SIZE - last_dma_pos_uart1 + dma_pos);

        if (len > 0)
        {
            if (last_dma_pos_uart1 + len <= UART_RX_BUFFER_SIZE) {
                FIFO_Write(&uart1_fifo, &uart1_dma_rx_buf[last_dma_pos_uart1], len);
            } else {
                uint16_t first_chunk = UART_RX_BUFFER_SIZE - last_dma_pos_uart1;
                FIFO_Write(&uart1_fifo, &uart1_dma_rx_buf[last_dma_pos_uart1], first_chunk);
                FIFO_Write(&uart1_fifo, &uart1_dma_rx_buf[0], len - first_chunk);
            }
            if (UART1TaskHandle != NULL) {
                BaseType_t xHigherPriorityTaskWoken = pdFALSE;
                xSemaphoreGiveFromISR(xUART1Sem, &xHigherPriorityTaskWoken);
                portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
            }
        }
        last_dma_pos_uart1 = dma_pos;
    }
    HAL_UART_IRQHandler(&huart1);
}
```

4.2. Buffer UART RX

Bộ đệm theo nguyên tắc “FIFO” chứa byte được nhận từ ISR UART RX. Để tạo ra bộ đệm FIFO cho các dữ liệu được nhận qua UART1/UART2 sử dụng thư viện tự FIFO viết (ngôn ngữ lập trình C).

4.3. Task UART Rx Parser

Task UART Rx Parser (của UART1/UART2) chờ semaphore từ ISR để phát hiện có dữ liệu mới, sau đó đọc các byte từ FIFO (`uart1_fifo`), phân tích nội dung bản tin MAVLink bằng hàm `mavlink_parse_byte()` được định nghĩa trong thư viện MAVLink V1 tự viết.

Nếu hợp lệ, payload của bản tin sẽ được trích xuất và đóng gói thành chuỗi “UARTx: <payload>” ($x = 1, 2$), rồi đưa vào hàng đợi (queue) để Task UART3 Tx truyền tiếp lên terminal trên PC.

Pseudocode minh họa 2 – Task UART Rx Parser

```
void StartUART1Task(void *argument)
{
    static mavlink_message_t mav_msg;
    static mavlink_state_t mav_state = MAVLINK_STATE_IDLE;
    QUEUE_t msg;
    uint8_t byte;

    for (;;)
    {
        xSemaphoreTake(xUART1Sem, portMAX_DELAY);

        do {
            int available = FIFO_Available(&uart1_fifo);
            for (int i = 0; i < available; i++)
            {
                FIFO_Read(&uart1_fifo, &byte, 1);

                if (mavlink_parse_byte(&mav_state, &mav_msg, byte))
                {
                    int payload_len = mav_msg.len;
                    char payload_str[256];
                    memcpy(payload_str, mav_msg.payload, payload_len);
                    payload_str[payload_len] = '\0';

                    msg.len = snprintf((char*)msg.data, sizeof(msg.data), "UART1: %s\r\n", payload_str);
                    osMessageQueuePut(UARTQueueHandle, &msg, 0, 20);
                }
            }
        } while (FIFO_Available(&uart1_fifo) > 0);
    }
}
```

4.4. Queue UART3 Tx

Sử dụng cơ chế hàng đợi (queue) trên CMSIS-RTOS V2 để nhận các nội dung payload trong các gói tin MAVLink V1 sau khi được parse trong Task UART Rx Parser của cả UART1 và UART2. Những nội dung payload này sẽ được xử lý trong Task UART3 Tx và truyền lần lượt lên terminal theo nguyên tắc FIFO.

4.5. Task UART3 Tx

Task UART3 Tx liên tục nhận bản tin từ hàng đợi (queue), nháy LED kiểm tra hoạt động, rồi truyền dữ liệu qua UART3 bằng DMA. Task chờ DMA hoàn tất bằng công cụ đồng bộ semaphore xUART3TxDoneSem trước khi kết thúc vòng lặp.

Pseudocode minh họa 3 – Task UART3 Tx

```
void StartUART3Task(void *argument)
{
    QUEUE_t msg;

    for (;;)
    {
        if (osMessageQueueGet(UARTQueueHandle, &msg, NULL, osWaitForever) == osOK)
        {
            HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
            if (HAL_UART_Transmit_DMA(&huart3, msg.data, msg.len) == HAL_OK)
            {
                xUART3TxDoneSem--;
            }
        }
    }
}
```

```

        xSemaphoreTake(xUART3TxDoneSem, portMAX_DELAY);
    }
}
}

```

4.6. Txcppltcallback

Khi DMA hoàn tất việc truyền dữ liệu qua UART3, ngắt Txcppltcallback này được gọi để giải phóng semaphore xUART3TxDoneSem, giúp Task UART3 Tx biết đã truyền xong và tiếp tục xử lý những bản tin kế tiếp.

Pseudocode minh họa 4 – Txcppltcallback

```

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART3)
    {
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;
        xSemaphoreGiveFromISR(xUART3TxDoneSem, &xHigherPriorityTaskWoken);
        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

```

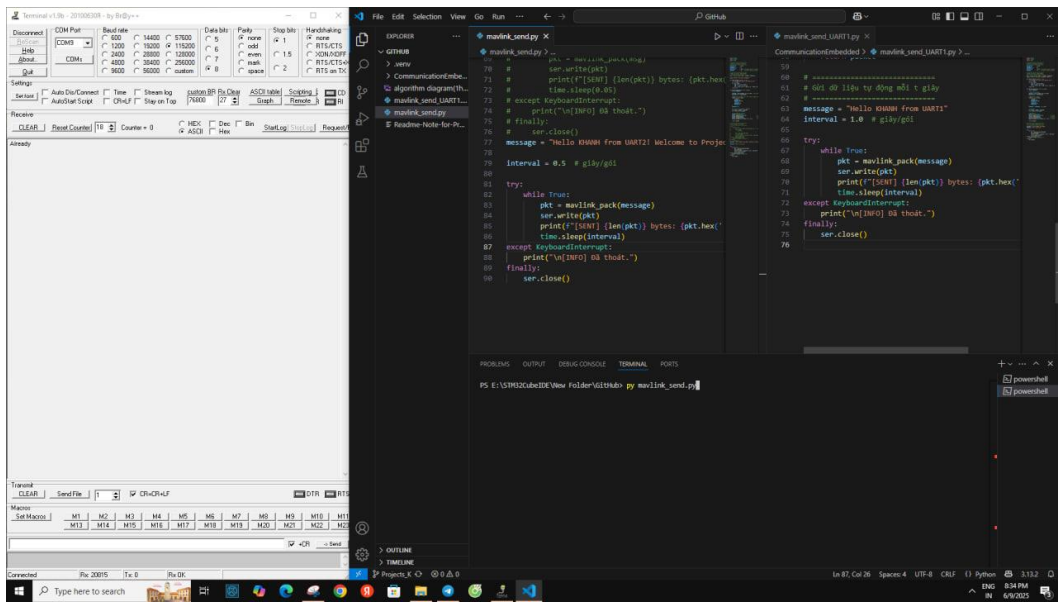
5. Chi tiết xử lý MAVLink V1

Mỗi bản tin MAVLink V1 được bắt đầu bằng byte 0xFE, gồm header (5 byte), payload (0 – 255 byte) và checksum (2 byte). Task UART Rx Parser sử dụng thư viện MAVLink V1 tự viết để parse gói tin MAVLink V1 được tạo và gửi qua các cổng UART1/UART2.

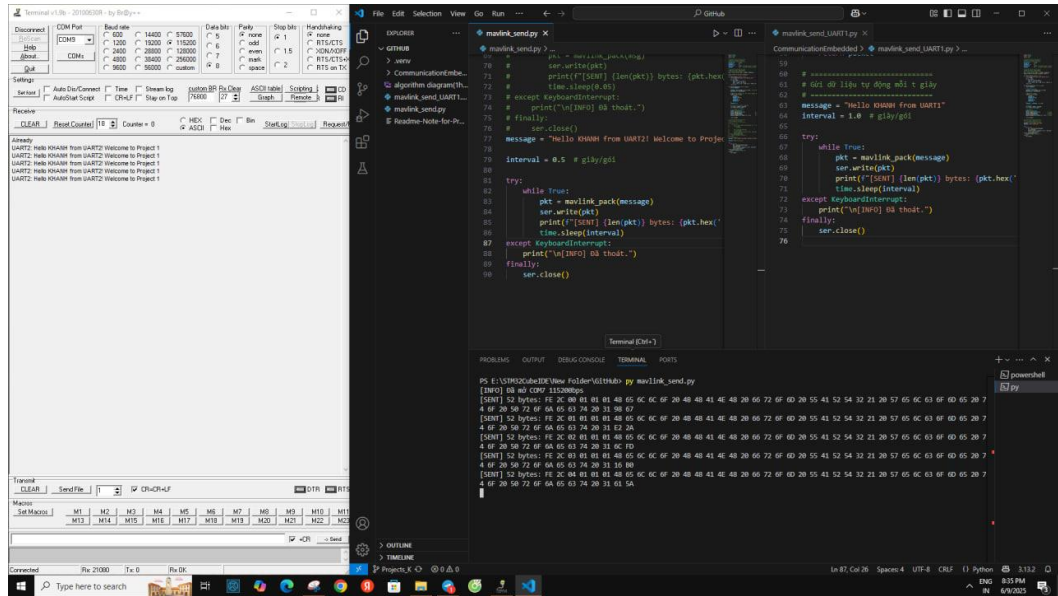
Trước đó, việc đóng gói tin MAVLink V1 được thực hiện tại các chương trình trong Visual Studio Code (sử dụng ngôn ngữ lập trình python).

6. Kết quả thử nghiệm hoạt động

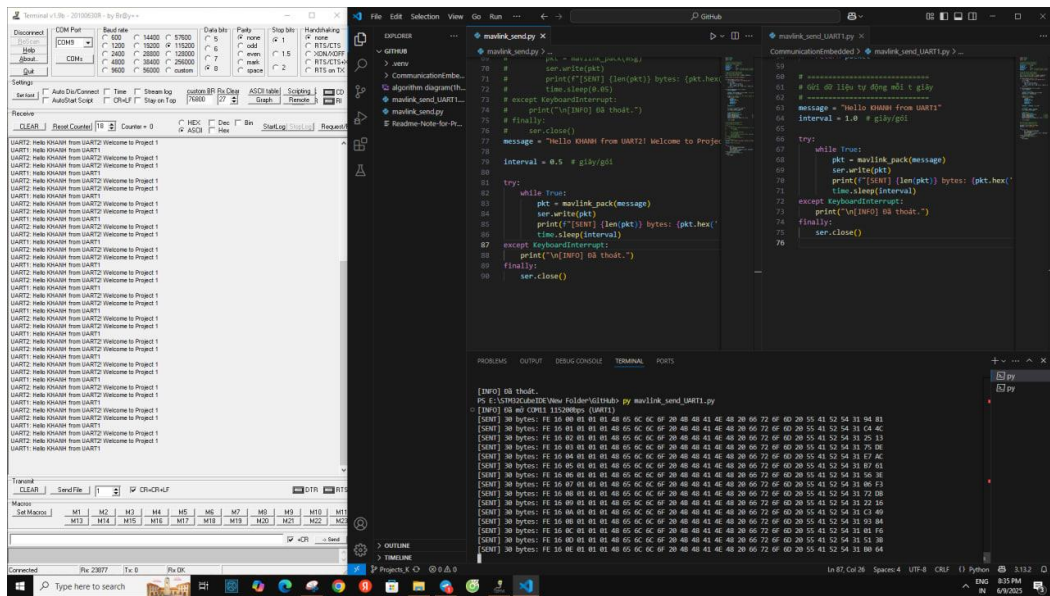
Sau khi lắp đặt thiết bị và nạp code như mô tả ở mục 2.2, thực hiện kiểm tra hoạt động của thiết bị và nhận được các kết quả như sau:



Hình 5 – Kết quả thử nghiệm khi bật thiết bị



Hình 6 – Kết quả thử nghiệm khi truyền tin MAVLink vào cổng UART2



Hình 7 – Kết quả thử nghiệm khi truyền tin MAVLink vào 2 cổng UART1 và UART2

Dựa vào những kết quả thu được có thể rút ra kết luận: thiết bị “Khởi tổng hợp dữ liệu” hoạt động đúng theo yêu cầu đã được đưa ra.

7. Một số lưu ý khác

Khi triển khai dự án cần lưu ý đến những yêu cầu sau để tránh phát sinh lỗi khi thiết bị hoạt động:

- những kích thước bộ đệm quan trọng được sử dụng như: UARTxTaskBuffer[] (x = 1, 2, 3); data[] (trong Queue); FIFO_Buffer để tránh tràn bộ nhớ.

- việc sử dụng các công cụ đồng bộ để kiểm soát truy cập dữ liệu chung của các Task thực hiện song song.

- hạn chế việc sử dụng “Polling” trong các Task và ISR.

8. Kết luận

Dự án lập trình nhúng “Khởi tổng hợp dữ liệu” đã được triển khai thành công với việc tích hợp hiệu quả các công nghệ UART, DMA, ISR và hệ điều hành thời gian thực FreeRTOS (CMSIS-RTOS V2). Hệ thống đáp ứng đúng yêu cầu: tiếp nhận dữ liệu từ hai kênh UART đầu vào, nhận dạng và kiểm tra bản tin MAVLink V1, sau đó truyền tiếp bản tin hợp lệ qua kênh UART đầu ra.

Trong quá trình thực hiện, đã đối mặt với nhiều thách thức khi làm quen với mô hình đa tác vụ và lập trình thời gian thực. Tuy nhiên, chính những trở ngại này đã mang lại nhiều bài học thực tiễn quý báu về xử lý đồng thời, phân tách chức năng, cũng như cách kiểm soát lỗi trong môi trường RTOS.

Ngoài ra, dự án không chỉ đạt được mục tiêu kỹ thuật đề ra, mà còn đóng vai trò như một bước đệm để làm quen với các quy trình thiết kế hệ thống nhúng trong thực tiễn – từ khâu kiến trúc phần mềm đến xử lý dữ liệu thực tế, đặc biệt hữu ích trong các ứng dụng đòi hỏi tính ổn định và thời gian thực cao như hàng không, tự động hóa hay robot.